```
$ docker build -t railsapp .
Sending build context to Docker daemon  102.9kB
«...»
Successfully built 577a1a5a2d2c
Successfully tagged railsapp:latest
```

The size of the build context is reported in the output—102.9 KB—which is smaller than prior to adding the Docker .dockerignore (138.8 KB). The savings will increase over time, especially as the Git history gets larger.

The Image Build Cache

During development, we rebuild our image fairly regularly, either to install new gems (bundle install is one of the steps in our Dockerfile) or to update our dependencies such as Node.js.

Just like a fast test suite helps by reducing the feedback loop, it's important to keep our image builds as fast as possible too. One way that Docker helps is by caching each step in our build, meaning that it only needs to rebuild from the first instruction in the Dockerfile where there is a change. A change could either be literally deleting or modifying the Dockerfile instruction, or it could be associated with filesystem changes, as we'll see shortly.

Since there are no changes to our Dockerfile or files since our last build, it would be extremely fast to rebuild the image now. Each step has already been built and cached, so Docker will have to do very little work.

Let's try rebuilding our image now:

```
$ docker build -t railsapp .
Sending build context to Docker daemon  102.9kB
Step 1/7 : FROM ruby:2.6
 ---> f28a9e1d0449
Step 2/7 : RUN apt-get update -yqq
 ---> Using cache
 ---> 761da319d69a
Step 3/7 : RUN apt-get install -yqq --no-install-recommends nodejs
 ---> Using cache
 ---> 145b025f550c
Step 4/7 : COPY . /usr/src/app/
 ---> Using cache
 ---> 045a92afdc82
Step 5/7 : WORKDIR /usr/src/app
 ---> Using cache
 ---> 1d89cb7f0720
Step 6/7 : RUN bundle install
 ---> Using cache
 ---> 81ad2d531548
```